

Post-Trade Review

Turning a month of order history into habits worth changing

01 One-pager	p. 2
02 Prompts used	p. 4
03 Evals	p. 9
04 Working app	p. 13

Submitted for the Nubra Product Management Intern take-home assignment
All four artifacts verified against ground_truth_patterns.csv and ground_truth_clean_control.csv

One-Pager

Your take on the problem statement

Most retail traders never look back at their own trade history — not because they lack the data, but because reviewing it is tedious, and because it can be uncomfortable to confront. The brief asks for a tool that turns a month of order history into 2-3 habits costing the trader money. The sharper version of this problem: it's not a reporting problem, it's a noticing problem. A trader who is cutting winners early and holding losers for a week doesn't need more data — brokers already show P&L per trade. They need someone to connect the dots across trades and say it back to them in a way that feels true, not generic.

I scoped this for an active retail trader (20-100 trades/month, mostly intraday/swing equity) rather than a buy-and-hold investor, since behavioral patterns like revenge trading or overtrading only show up at that frequency. I focused on three specific, detectable habits: the disposition effect (cutting winners early, holding losers), revenge trading (oversized trades right after a loss), and overtrading (clusters of low-conviction, near break-even trades). These were chosen because they're identifiable from order timestamps and P&L alone — no need for the trader's intent or a broker integration to detect them.

Why Nubra should solve it

Nubra's revenue is tied to trading activity, so there's a natural tension: a tool that makes traders trade less isn't obviously in the platform's short-term interest. But retail trading platforms increasingly compete on trust, not just execution speed or UI — traders who feel a platform is actually looking out for them are more likely to stay active for years rather than blow up an account in months and churn. A post-trade review tool is a low-cost way to build that trust, and it differentiates Nubra from platforms that only optimize for order volume.

The solution space

I considered a dashboard of trading metrics (win rate, avg P&L, Sharpe-like stats) but rejected it — the brief explicitly warns against “another dashboard,” and metrics without narrative rarely change behavior. Instead, the tool takes a raw order-history CSV, reconstructs round-trip trades, checks each trade set against explicit, threshold-based rules for the three habits above, and returns a short plain-language report: at most 3 findings, each with one real number as evidence and one concrete rule to try next month — not advice, a rule.

What is in scope

- One month of equity order history (CSV upload)
- Detecting the three named patterns only, with explicit numeric thresholds
- A short, non-judgmental report (max 3 findings) with one adoptable rule each

What is out of scope

- Live broker integration or real-time data
- F&O-specific behavior (expiry-driven decisions, Greeks)
- Cross-month tracking of whether a habit actually improved
- Benchmarking a trader against other users — too sensitive, and not needed to make the insight land
- Automated actions (e.g., auto stop-losses) — this is a review tool, not an execution tool

02 PROMPTS USED

Prompts Used

This document shows the actual prompt iterations, including the ones that didn't work, in the order I used them.

Attempt 1 — naive, too vague (didn't work well)

You are a trading coach. Here is a trader's order history for one month in CSV format. Look at the trades and tell the trader what they're doing wrong and how to improve.

CSV:
{csv_data}

Why this failed

- The CSV is order-level (individual BUY/SELL rows), not trade-level. The model tried to comment on individual orders instead of matching BUY+SELL pairs into round-trip trades first — so its “insights” were often about single legs of a trade, which don't mean anything on their own.
- “Tell the trader what they're doing wrong” produced generic, judgmental output (“you should be more disciplined”) with no specific numbers — not usable.
- No constraint on how many findings, so it sometimes returned 8-10 shallow observations instead of 2-3 sharp ones.
- No definition of what patterns to look for, so it invented ad hoc ones each run — output wasn't consistent or reproducible.

Attempt 2 — added structure, still missing guardrails

You are analyzing a trader's order history to find behavioral patterns.

Step 1: Match BUY and SELL rows for the same symbol into round-trip trades (entry time, exit time, holding duration, P&L).

Step 2: Look for these three patterns:

- Disposition effect: winning trades closed quickly, losing trades held much longer.
- Revenge trading: a much larger trade placed shortly after a loss.
- Overtrading: clusters of many trades with very short holding times, mostly breakeven.

Step 3: Report the 2-3 patterns that show up most clearly, each with a specific number (e.g. average holding time), and one concrete rule the trader could adopt.

CSV:

{csv_data}

Why this was still not good enough

- Better structure, but the model would sometimes still assert a pattern with weak or borderline evidence, because there was no explicit threshold for what counts as “shortly after,” “much larger,” or “much longer.”
- Tone was inconsistent — some runs sounded clinical/dashboard-like, which is exactly the “another dashboard” outcome the brief warns against.
- No instruction to abstain when a pattern isn't clearly present, so on a cleaner-than-usual month, the model would still force 2-3 findings.

Attempt 3 — final prompt-only version used in the tool

You are a plain-spoken trading coach reviewing one month of a retail trader's order history. Your job is to help them notice a real, specific pattern in their own behavior — not to lecture them, rank them, or produce a dashboard of metrics.

INPUT

You will receive a CSV of raw orders: order_id, symbol, side (BUY/SELL), quantity, price, timestamp, trade_type.

STEP 1 — Reconstruct round-trip trades

Match each BUY with its corresponding SELL for the same symbol and quantity to form a completed trade. For each, compute: entry time, exit time, holding duration (in hours), and P&L.

STEP 2 — Check for these three patterns only. Do not invent others.

1. Disposition effect

Compare average holding time of winning trades vs. losing trades.

Only flag this if losing trades are held on average at least 2x longer than winning trades, AND there are at least 3 trades of each type to compare.

2. Revenge trading

Look for a losing trade followed, within 60 minutes, by a new trade that is at least 1.5x the quantity of the trader's average trade size.

Only flag this if it happens at least 3 times in the month.

3. Overtrading

Look for days with at least 4 trades where the average holding time is under 30 minutes and average P&L per trade is close to zero (between -1% and +1% of position value).

Only flag this if this happens on at least 2 different days.

If a pattern does not meet its threshold, do not report it.

STEP 3 — Output format

Return at most 3 findings. For each: one sentence naming the pattern in plain language (no jargon), one sentence with the specific number(s) that show it,

and one sentence suggesting a single, concrete rule the trader could adopt next month — not generic advice.

Tone: constructive and specific, never clinical or judgmental, and never comparing them to other traders. Do not output a table or a dashboard.

CSV:
{csv_data}

Why this version works

- Explicit thresholds remove ambiguity and reduce hallucinated patterns.
- Forcing round-trip reconstruction as Step 1 fixes the single-leg confusion from Attempt 1.
- The “do not invent others” and “only if threshold met” instructions stop the model from forcing findings when the data doesn't support it.
- The output-format constraints directly address the brief's “not another dashboard” requirement.
- Asking for one concrete rule per finding — rather than “advice” — is what turns this from a report into something actionable.

Attempt 4 — moved the arithmetic out of the prompt entirely (post-ship fix)

After wiring Attempt 3 into a real API call against the actual monthly tradebook (52 reconstructed trades), two failures showed up that no amount of prompt tightening had caught, because they only appear once the model is actually asked to do the arithmetic at real scale:

- Wrong numbers, stated confidently. The model reported a 4.7x disposition ratio (“avg 46.3h vs 9.9h”) when the true ground-truth ratio, independently computed from `ground_truth_patterns.csv`, is 13.2x (“avg 20.4h vs 1.5h”). The model's pattern logic was fine; its arithmetic over 50+ trades in a single completion was not.
- Preamble breaking JSON, even under explicit instruction not to. Despite “respond with ONLY valid JSON, no preamble” in Attempt 3, the model sometimes still opened with a sentence like “I'll work through this...” before the JSON, which broke parsing outright.

Rather than keep prompting around these two failures, the architecture changed so the model can no longer cause either one:

- Parse the CSV and reconstruct round-trip trades in JavaScript — deterministic, since this tradebook format uses one `order_id` per round-trip.
- Check all three pattern thresholds in JavaScript, using the exact same numeric rules as Attempt 3.
- Only call the model with the already-confirmed patterns and their exact numbers, asking it to write a plain-language observation and a concrete rule per pattern — explicitly instructed not to recompute, round, or invent any number.

Final prompt used for the copywriting step (much smaller than Attempt 3's, by design)

You are a plain-spoken trading coach. The pattern(s) below have ALREADY been detected and verified in a retail trader's order history this month — the numbers are confirmed correct. Do not recompute, restate differently, round, or invent any numbers; only use the exact figures given to you.

CONFIRMED PATTERN(S) THIS MONTH:

- disposition: losing trades held 13.2x longer than winners (avg 20.4h vs 1.5h, across 24 losing and 28 winning trades)
- revenge: 5 times this month, a losing trade was followed within 60 minutes by a new trade at least 1.5x your average size (average trade size: 37 shares)
- overtrading: on 3 separate days this month you placed 4+ trades in a row averaging under 30 minutes holding time with close-to-breakeven P&L (15 trades total)

For each pattern listed above, by its key name, write:

- "observation": one sentence naming the pattern in plain language, no jargon (never say "disposition effect" — describe the behavior itself)
- "rule": one single, concrete rule the trader could adopt next month — not generic advice like "be more disciplined"

Tone: constructive and specific, like a coach who actually looked at their trades — never clinical, judgmental, or comparing them to other traders.

Respond with ONLY valid JSON, first character must be {, no markdown fences, no preamble, no commentary.

Why this works better than a bigger or stricter prompt

- The model's job shrank from “reconstruct 50+ trades, run three statistical comparisons, and write prose — all correctly, in one pass” to “write two sentences about a fact I'm handing you.” Numeric errors become structurally impossible rather than merely instructed against.
- JSON parsing became far more reliable — the model only wraps a few short sentences, rather than reasoning through an entire dataset first.
- It also surfaced two bugs in my own detection code that only testing against ground truth caught — see Section 5 of the evals document. Moving logic out of the model and into code only makes it trustworthy if the code itself is then checked against ground truth — “no longer the model's problem” isn't the same as “correct.”

Deployment note (not a prompt change — how the prompt gets called)

Attempt 4's prompt text is exactly what ships — nothing below changes the words sent to the model. What changed afterward is how and whether that prompt gets called, once the app needed to run on a free host rather than only inside claude.ai (public sharing of a claude.ai artifact required a paid plan, which isn't guaranteed for whoever opens this). The app now tries, in order:

- Claude's sample capability — if opened through its claude.ai artifact link, it asks the viewer's own Claude account to run Attempt 4's prompt. No key needed, but only works on that one link.

- A user-supplied Anthropic API key — if hosted elsewhere (Netlify/Vercel/GitHub Pages/local) and someone pastes in their own key, the exact same Attempt 4 prompt is sent directly to the Anthropic API from the browser. The key lives only in memory for that session.
- Fixed template phrasing — if neither of the above is available, the app skips the model call entirely and uses hand-written sentences built around the same code-verified numbers. No prompt runs at all in this path.

Every path uses the identical pre-computed, ground-truth-verified numbers; only the sentence-writing step differs. This means the tool never dead-ends regardless of where it's hosted or whether the reviewer has an API key — worst case, they see accurate findings in template English rather than AI-written English, never an error screen.

Notes for next iteration (superseding Attempt 3's)

- The order_id-pair matching in Attempt 4's code assumes exactly one BUY and one SELL leg per order_id — correct for this dataset's format, but a real tradebook with partial fills or multiple concurrent lots of the same symbol would need proper FIFO lot matching.
- Boundary conditions (holding ratio exactly at 2x, a gap exactly at 60 minutes) haven't been explicitly tested against the code thresholds.

Evals

“Good enough to ship” for this tool means: every finding it surfaces is true, specific, and actionable — and it says nothing when there's nothing to say. A tool that invents a pattern to fill a quota is worse than useless for a product built on trust, so precision matters more than recall here.

I tested against two datasets: `tradebook_august_2025.csv` (the engineered dataset with 3 known patterns, scored against `ground_truth_patterns.csv`), and `tradebook_clean_control.csv` (an independently generated month with no injected patterns, scored against `ground_truth_clean_control.csv`) — the negative control, testing whether the tool forces findings on a boring month.

1. Detection accuracy (against known patterns)

Check	Pass condition
Recall	At least 2 of the 3 injected patterns are correctly surfaced from <code>tradebook_august_2025.csv</code>
Precision	No pattern is claimed that isn't backed by real trades in the ground truth
Negative control	On <code>tradebook_clean_control.csv</code> , the tool reports 0 findings or explicitly states no strong pattern was found

I ran the prompt against both files and manually cross-checked each claimed finding against the ground-truth tags — e.g. if it says “revenge trading,” I check that the specific trades it's implicitly describing are actually tagged `revenge_trigger_loss` / `revenge_trade_oversized`, not just any two trades that happened to be near each other in time.

2. Numerical accuracy (no invented numbers)

For every number the tool cites, I independently computed the real value from the CSV using a short Python script against `ground_truth_patterns.csv` and checked it matched exactly, not just approximately.

Example check used on the August dataset

- Ground truth: avg holding time for losing vs. winning trades, computed directly from `ground_truth_patterns.csv` → 20.4h vs 1.5h, a 13.2x ratio.
- Tool's claim (final version): “losing trades held 13.2x longer than winners (avg 20.4h vs 1.5h, across 24 losing and 28 winning trades)” — exact match.
- Tool's claim in an earlier version: “you held losing trades roughly 4.7x longer” (46.3h vs 9.9h) — this failed the check and is exactly why the architecture changed (see Section 5).

Any number that doesn't check out is an automatic fail — a specific but wrong number is more damaging to trust than a vague true statement.

3. Format and tone compliance

- 3 or fewer findings
- No table, metrics list, or dashboard-style layout
- No jargon (e.g. “disposition effect”) stated without being translated into plain behavior language
- Each finding includes one specific number from real data
- Each finding includes exactly one concrete, adoptable rule — not generic advice
- Tone reads as constructive, not judgmental or comparative to other traders

This is a manual checklist rather than an automated one — tone is the hardest part to grade programmatically, and at this stage a human read is more reliable than trying to build a sentiment classifier for a 4-6 hour build.

4. Edge cases specifically tested

- Clean month (no real pattern): does it force findings anyway, or correctly say there's not much to flag? This is the single most important test, since over-eager pattern-finding is the failure mode most likely to erode a trader's trust. Result: 0 findings, correctly.
- Borderline pattern: manually edited a couple of trades in the ground truth to sit just below the disposition-effect threshold (loser held 1.8x longer, threshold is 2x) to check the tool doesn't flag it anyway — testing whether the explicit thresholds are actually respected, not just suggested.

5. Iteration after initial ship attempt — two rounds of bugs, caught by testing against ground truth

The first working version put trade reconstruction, threshold-checking, and prose generation all inside a single model call (Attempt 3). Running it against the real 52-trade tradebook — not just imagining the prompt would hold — surfaced a real numeric error: it reported a 4.7x disposition ratio (“avg 46.3h vs 9.9h”) against a true ground-truth ratio of 13.2x (“avg 20.4h vs 1.5h”). The model's pattern logic was fine; its arithmetic over 50+ trades in a single completion was not. Separately, even under an explicit “JSON only, no preamble” instruction, the model sometimes still opened with a sentence like “I'll work through this...”, which broke JSON parsing outright.

Fix: moved trade reconstruction and threshold-checking entirely into deterministic code, and reduced the model's job to writing plain-language prose around numbers that are already computed and verified — never asking it to compute or restate a number itself.

That rewrite then surfaced two further bugs — this time in my own detection code, not the model — caught only by testing the code's actual output against `ground_truth_patterns.csv` line-by-line rather than eyeballing plausibility:

Pattern	First code version	Root cause	Fixed / ground truth
Revenge trading	6 instances	One oversized trade was matched against two different prior losses (double-counted)	5 instances — matches ground truth
Overtrading	17 trades / 3 days	2 trades that were genuinely revenge trades also happened to be quick, and got double-counted	15 trades / 3 days — matches ground truth

Both fixes were verified by running the app's actual parsing and detection code — not a re-implementation — directly against both CSVs outside the browser, independent of the UI. Final result: disposition 13.2x, revenge 5, overtrading 15/3 days, all exact matches to ground truth; clean control still returns zero findings.

Takeaway for the eval process itself

Moving logic out of the model and into code only makes it trustworthy if the code is then checked against ground truth just as rigorously as the model's output was. “No longer the model's problem” is not the same as “correct” — it just relocates where the bug can hide.

6. Deployment resilience — works with zero setup, on any host

Public sharing of a `claude.ai` artifact link turned out to require a paid plan, which isn't something I can guarantee the reviewer has — so the tool needed to work correctly on a free static host with no backend and no guaranteed API access. That changes what “ship” needs to mean: not just correct output, but correct output regardless of where the AI-writing step is or isn't available.

The app now tries three paths, in order, every time it needs to turn a confirmed pattern into a sentence:

- Claude's built-in sample capability — works only when opened through its `claude.ai` artifact link, using the viewer's own Claude access, no key needed.
- A user-supplied Anthropic API key, entered in an optional field — works on any free host, called directly from the browser, kept in memory only for that session.
- Fixed, hand-written template phrasing — the default on a fresh host with no key entered. No model call happens in this path at all.

The test that matters here: every path uses the exact same pre-computed, ground-truth-verified numbers — only the sentence-writing step differs between them. I verified this by disabling `window.claude` and leaving the API key blank (simulating the worst case) and confirming the app still returns correct, readable findings via the template path rather than an error screen. “Good enough to ship” now explicitly includes: never dead-ends into a blank error, regardless of hosting.

What “ship” means at this stage

For a 4-6 hour take-home prototype, “ship” means: on the engineered dataset it correctly finds the injected patterns with numbers that exactly match ground truth, and on the clean control it does not fabricate findings. It does not mean this is robust to messy real-world tradebooks (partial fills, multiple concurrent lots, F&O contracts) — that’s explicitly flagged as future work below, not silently assumed to work.

What I'd add with more time

- Proper FIFO lot matching instead of the current order_id-pair matching, to handle partial fills and multiple concurrent lots of the same symbol.
- Explicit boundary-condition tests against the code thresholds (holding ratio exactly at 2x, a gap exactly at 60 minutes).
- A larger set of control datasets (5-10 months, varying degrees of messiness) to get an actual precision/recall percentage rather than a single pass/fail run.
- User-facing eval: show 3-5 real traders the output and ask “does this feel true and useful, or generic and annoying” — the tone checklist above is my best guess at what that would surface, not a replacement for it.

Working App

Post-Trade Review — a tool that turns a month of order history into 2-3 specific, verified trading habits worth changing.

Live link

<https://dulcet-selkie-54917a.netlify.app>

Try it: click “a typical month” or “a clean month” on the page — no upload needed.

How it works

Trade reconstruction and all pattern-matching numbers are computed deterministically in JavaScript and verified against `ground_truth_patterns.csv` (see Evals, Sections 1-2 and 5). The plain-language phrasing around those numbers is written by Claude when available, with a fixed-text fallback otherwise — so the tool never shows an error screen regardless of hosting (see Evals, Section 6, and the Prompts document's “Deployment note”).

Hosted on Netlify as a single self-contained HTML file — no backend, no signup required to view it.